

FluidsFormer: A Transformer-Based Approach for Continuous Fluid Interpolation

Bruno Roy
bruno.roy@autodesk.com
Autodesk Research
Montreal, Canada

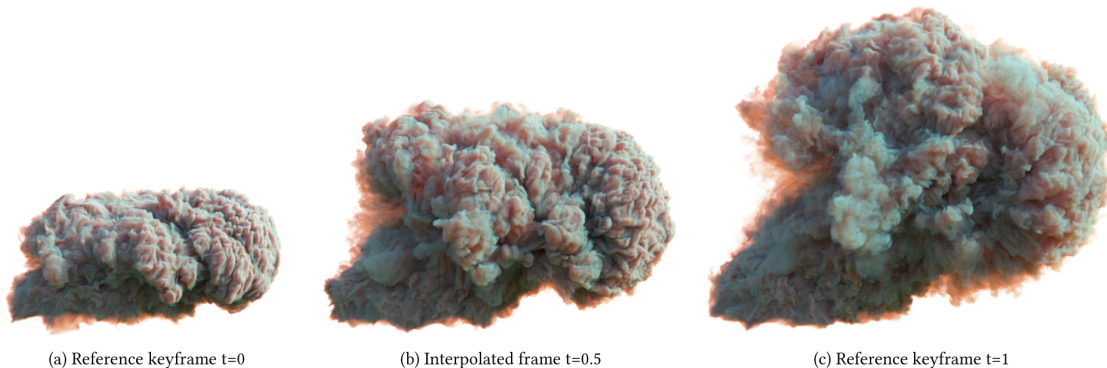


Figure 1: Given input keyframes, our approach interpolates substeps of a fluid simulation – resulting in a smooth and realistic animation.

ACM Reference Format:

Bruno Roy. 2024. FluidsFormer: A Transformer-Based Approach for Continuous Fluid Interpolation. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Posters (SIGGRAPH Posters '24)*, July 27–August 01, 2024, Denver, CO, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3641234.3671085>

1 INTRODUCTION

In recent years, there has been a growing interest in editing within the latent space, posing a significant challenge in delivering controllable data-driven capabilities. Among many other areas, physics-based animation remains particularly challenging to edit as it relies on complex physics rules and principles that need to be satisfied for realism. Another challenge posed by the VFX industry is the need to adapt these principles to align with art direction, as observed in animation films. Striking a balance between realism and controllability poses difficulty in providing tools for editing natural phenomena such as fluids. For several decades, numerous researchers have endeavored to enhance the controllability and flexibility of fluid editing – spanning from local editing of keyframes [Pan et al. 2013] to flow-based methods [Roy et al. 2021; Sato et al. 2018]. Our approach shares similarities with that of [Roy et al. 2021] as we aim

to interpolate fluids in a data-driven manner using the advection scheme of Eulerian simulations.

2 OUR METHOD

Although Transformer-based networks have been primarily introduced for natural language processing (NLP) and text generation, they offer interesting properties for sequential data in general. In our method, we propose to leverage an attention-based architecture within a continuous-time framework to learn and interpolate simulation properties per frame in an approximate analytical manner. In the following sections, we will outline the details spanning from data preparation to network architecture and training using a transformer-based encoder-decoder network. We will also highlight a few concrete use cases to introduce novel ways of generating and editing fluids. The data preparation is divided into two main steps: (1) generating the temporal embeddings for the fluid element’s states and (2) handling the tokenization process in a physics-adapted context. The tokenization process is performed by parsing and splitting the Navier-Stokes equation into components (see Eq. 1) – we intentionally omit the viscosity term to simplify the related operations.

$$\rho \left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} \right) = -\nabla p \quad (1)$$

We learn the latent embedding of the advection part of the governing equation using standard multi-head self-attention blocks from the PITT architecture [Lorsung et al. 2024]. That way, our model is capable of learning to interpolate physics properties analytically (e.g., densities ρ). As we do not consider the equilibrium equation (i.e., $\nabla \cdot \mathbf{u} = 0$), we handle the volume preservation part by simply

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGGRAPH Posters '24, July 27–August 01, 2024, Denver, CO, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0516-8/24/07

<https://doi.org/10.1145/3641234.3671085>

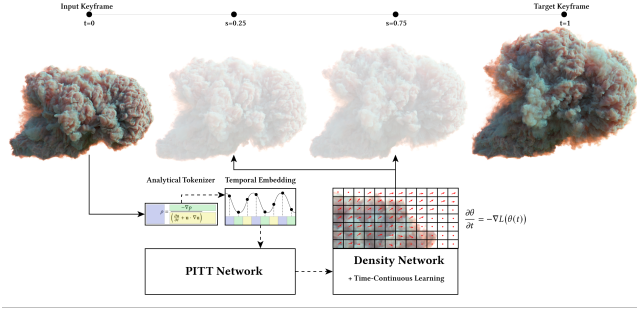


Figure 2: Here is an overview of the *FluidsFormer* approach.

penalizing in our loss function the solutions diverging too much from the reference.

2.1 Network Architecture

Our network architecture is composed of two stacked networks: (1) the pre-trained PITT network (transformer-based network) for solving the governing partial differential equation of the fluid dynamics and (2) the density network (RNN) to learn and predict the time-continuous density for the substeps between the input keyframes. We use the 18-layer architecture for our residual neural network as it gives us a decent performance (i.e., training/inference speed and accuracy) while reducing the requirement for an enormous dataset.

2.2 Training and Dataset

The density network is trained on normalized data $[-1, 1]$ outputted from the PITT network. During training, each simulation scenario is processed through the PITT network to generate the latent embedding of the non-viscous governing equation and provides an analytically-driven approximation to our density network to predict the correct density in space and time. To update the parameters, we use a Huber loss function L_δ minimizing a single term considering the possible outliers (with some regularization term): the difference in density between the ground truth ρ and the prediction $\hat{\rho}$.

$$L_\delta(\rho, \hat{\rho}) = \begin{cases} \frac{1}{2}(\rho - \hat{\rho})^2 & |\rho - \hat{\rho}| \leq \delta, \\ \delta|\rho - \hat{\rho}| - \frac{1}{2}\delta^2 & \text{otherwise.} \end{cases} \quad (2)$$

As the reference density is advected using the divergence-free velocity field, we already the volume preservation law.

We validated and tweaked the hyperparameters of our model using a 2D dataset of laminar and turbulent flows generated with OpenFOAM [Jasak 2009]. Then we generated volumetric data from Eulerian simulations using Bifrost. The volumetric data points are constituted of a position (center of the cell), a velocity, and a density. The volumetric dataset is composed of 1000 (800 for training, 100 for validation, and 100 for testing) smoke inflow and emission simulations of 50 frames in length, without and with a single obstacle placed at random locations. As for most RNN architectures, we need a significant amount of data to properly generalize without dissipating the small-scale details in the simulation (e.g., second-order vorticity).

2.3 Continuous-Time Learning

Similarly to [Chen et al. 2024], our architecture uses a continuous-time multi-head attention module to transform time-varying sequence relationships into vectors of queries Q , keys K , and values V . As opposed to [Chen et al. 2024] and inspired by [Deleu et al. 2022], we formulate the learning algorithm to follow the input velocity during training – allowing our model to converge faster and to reflect the analytical framework as discussed in Sec. 2. The updated gradients are then used to update the parameters defining the fluid state. In other words, we train our model to evaluate the density based on the advection term and the velocities of the governing equation of the input system. An inherent advantage of our analytically-driven approach is that we can perform the inference part discretization-free.

3 VARIOUS APPLICATIONS

Our main goal with this approach is to propose a continuous-time transformer model capable of learning the underlying dynamics of fluid systems for interpolation purposes. The idea behind interpolating fluids is to generate a visually appealing and temporally smooth animation using only a few keyframes at large timesteps. Our interpolation methods will simulate the substeps between the provided reference keyframes (as shown in Fig. 1). We also take advantage of the generated tree structure to combine keyframes using CSG operations. Using the volumetric data (i.e., properly stored in grid cells), we can mix multiple keyframes into a single target in our approach and produce a completely new animation. Our approach is also capable of generating tree-based variants. For each frame, we output multiple probable solutions using top-k sampling along with diverse Beam Search on the decoder side to encourage diversity in the generated sequences. From this set of solutions, we build a tree structure that allows us to branch out at any node to produce variants of a single simulation while preserving the initial conditions.

4 CONCLUSION

While our approach still relies on a coarse numerical simulation, we are confident that it introduces novel and less-linear ways of interacting with fluids.

REFERENCES

- Yuqi Chen, Kan Ren, Yansen Wang, Yuchen Fang, Weiwei Sun, and Dongsheng Li. 2024. ContiFormer: Continuous-time transformer for irregular time series modeling. *Advances in Neural Information Processing Systems* 36 (2024).
- Tristan Deleu, David Kanaa, Leo Feng, Giancarlo Kerg, Yoshua Bengio, Guillaume Lajoie, and Pierre-Luc Bacon. 2022. Continuous-time meta-learning with forward mode differentiation. *arXiv preprint arXiv:2203.01443* (2022).
- Hrvoje Jasak. 2009. OpenFOAM: Open source CFD in research and industry. *International Journal of Naval Architecture and Ocean Engineering* 1, 2 (2009), 89–94.
- Cooper Lorusung, Zijie Li, and Amir Barati Farimani. 2024. Physics Informed Token Transformer for Solving Partial Differential Equations. *Machine Learning: Science and Technology* (2024).
- Zherong Pan, Jin Huang, Yiyang Tong, Changxi Zheng, and Hujun Bao. 2013. Interactive localized liquid motion editing. *ACM Transactions on Graphics (TOG)* 32, 6 (2013), 1–10.
- Bruno Roy, Pierre Poulin, and Eric Paquette. 2021. Neural upflow: A scene flow learning approach to increase the apparent resolution of particle-based liquids. *Proceedings of the ACM on Computer Graphics and Interactive Techniques* 4, 3 (2021), 1–26.
- Syuehei Sato, Yoshinori Dobashi, and Tomoyuki Nishita. 2018. Editing fluid animation using flow interpolation. *ACM Transactions on Graphics (TOG)* 37, 5 (2018), 1–12.